Guia de Estudos PIBITI

Mês 4 — Calibração e Primeiros Descritores (dez/2026)

Projeto de Pesquisa em Detecção de Deepfakes de Áudio

Dezembro de 2026

Sumário

1 O Que o Plano Exige **2**

2 Calibração do Nível de Decomposição **3**
2.1 Por que calibrar? 3
2.2 Código de Comparação 3

3 Calibração da Wavelet **5**
3.1 Código de Comparação de Wavelets 5

4 Descritores — O Que São? **6**

5 Energia **7**
5.1 Implementação da Energia 7

6 RMS (Root Mean Square) **8**
6.1 Implementação do RMS 8

7 ZCR (Taxa de Cruzamento por Zero) **9**
7.1 Implementação do ZCR 9

8 Envelope via Transformada de Hilbert **10**
8.1 Implementação do Envelope e Descritores 10

9 Combinando Descritores por Sub-banda **12**
9.1 O Extrator Final 12

10 Documentação de Parâmetros **13**
10.1 Dicionário de Configuração 13

11 Brincadeiras Interativas **14**

12 Checklist do Mês 4 **16**

13 Desafio Final do Mês 4 **17**

Capítulo 1

O Que o Plano Exige

Neste quarto mês do projeto PIBITI, nosso foco será a fixação dos parâmetros base da Wavelet (nível de decomposição e tipo da wavelet) e o início da extração de descritores de contorno de onda (*wave contour*).

Atividades e Objetivos:

- Calibrar o nível de decomposição L (começando por $L = 4 \rightarrow 2^L$ sub-bandas).
- Preparar a comparação de wavelets candidatas (db4, db8, sym8).
- Iniciar a extração de descritores por sub-banda: Energia, RMS (Root Mean Square) e ZCR (Zero Crossing Rate).
- **Prioridade:** manter o *core* em $L = 4$ e wavelet *db4* aplicados em 2 corpora centrais.

Atenção

O envelope de cada sub-banda deve ser calculado utilizando a Transformada de Hilbert (`scipy.signal.hilbert`), extraindo a magnitude do sinal analítico. A partir do envelope, extrairemos a média, o desvio padrão e a curtose. Todos os parâmetros (nível, wavelet, tipo de borda) devem ser documentados para garantir reprodutibilidade.

Marco / Entrega

Core nível/wavelet definido; sub-bandas reconstruídas + primeiros descritores (energia/RMS, ZCR, e estatísticas do envelope) implementados.

Capítulo 2

Calibração do Nível de Decomposição

2.1 Por que calibrar?

A escolha do nível L é um *trade-off*: níveis maiores (L elevado) geram mais sub-bandas, o que implica uma resolução de frequência mais fina; entretanto, isso resulta em tamanhos de sub-bandas cada vez menores, especialmente no domínio temporal, e aumenta o custo computacional.

Vamos comparar os níveis $L = 3$, $L = 4$ e $L = 5$:

- $L = 3$: $2^3 = 8$ sub-bandas.
- $L = 4$: $2^4 = 16$ sub-bandas (nosso nível alvo inicial).
- $L = 5$: $2^5 = 32$ sub-bandas.

2.2 Código de Comparação

```python
import pywt
import numpy as np
import time

def compare_levels(signal, wavelet='db4', levels=[3, 4, 5]):
    results = []
    for L in levels:
        start_time = time.time()
        # wp (wavelet packet)
        wp = pywt.WaveletPacket(data=signal, wavelet=wavelet, mode=
            'symmetric', maxlevel=L)
        nodes = [n.path for n in wp.get_level(L, 'freq')]
        end_time = time.time()

        results.append({
            'L': L,
            'Sub_Bands': len(nodes),
            'Len_Node': len(wp[nodes[0]].data),
            'Time_ms': (end_time - start_time) * 1000
        })
    return results
```

```python
# Gerando sinal de teste
fs = 16000
t = np.linspace(0, 1, fs)
signal = np.sin(2 * np.pi * 1000 * t) + np.random.randn(fs) * 0.1

res_levels = compare_levels(signal)
for r in res_levels:
    print(f"L={r['L']} | Bands={r['Sub_Bands']} | Size={r['Len_Node
        ']} | Time={r['Time_ms']:.2f}ms")
```

Decisão: O projeto começa com $L = 4$ como *core*. Ele oferece um bom equilíbrio com 16 bandas, sem comprometer demais o tamanho do vetor na extração de características.

Capítulo 3

Calibração da Wavelet

Temos diferentes famílias de wavelets. Vamos comparar as clássicas da família Daubechies (db4, db8) e Symlets (sym8). Aspectos a observar:

- **Tamanho do filtro:** db8 tem filtro mais longo que db4, podendo captar padrões diferentes mas espalhando os artefatos de borda.
- **Erro de Reconstrução:** Idealmente, deve ser próximo de zero em pacotes wavelet perfeitamente ortogonais.

3.1 Código de Comparação de Wavelets

```
def get_reconstruction_error(signal, wavelet, level):
    wp = pywt.WaveletPacket(data=signal, wavelet=wavelet, mode='
        symmetric', maxlevel=level)
    # Reconstruindo a partir de todos os n s folha
    nodes = [n.path for n in wp.get_level(level, 'freq')]
    reconstructed = wp.reconstruct(update=False)
    # Evita problemas de padding
    reconstructed = reconstructed[:len(signal)]
    error = np.max(np.abs(signal - reconstructed))
    return error
```

```
wavelets = ['db4', 'db8', 'sym8']
for w in wavelets:
    start_time = time.time()
    err = get_reconstruction_error(signal, w, level=4)
    dt = (time.time() - start_time) * 1000
    print(f"Wavelet: {w} | Max Error: {err:.2e} | Time: {dt:.2f}ms"
        )
```

Decisão: Adotaremos `db4` como o *core* inicial para os experimentos, expandindo posteriormente conforme os resultados em bases de dados maiores.

Capítulo 4

Descritores — O Que São?

Descritores, ou características (*features*), são valores numéricos extraídos de um sinal que o caracterizam de alguma forma. Eles reduzem a enorme dimensionalidade do áudio bruto (ex: 16000 amostras por segundo) a um pequeno vetor de números significativos.

Por que extrair por sub-banda? A Wavelet Packet Transform decompõe o sinal em faixas de frequência. Um som sintético de *deepfake* pode apresentar artefatos apenas nas altas frequências (sub-bandas finais). Extraindo os descritores para cada uma das 16 sub-bandas, nosso vetor de características ganha resolução no espaço de frequências e pode capturar as anomalias onde elas ocorrem.

Se extrairmos 6 descritores por banda, e temos 16 bandas, nosso *feature vector* terá $16 \times 6 = 96$ dimensões por segmento de áudio.

Capítulo 5

Energia

A **Energia** é simplesmente a soma dos quadrados das amostras do sinal de uma sub-banda. Fórmula: $E = \sum_{n=1}^{N} |x[n]|^2$

5.1 Implementação da Energia

```python
def calc_energy(x):
    """Calcula a energia total do sinal x."""
    return np.sum(np.square(x))
```

```python
# Testando com um vetor simples
x_test = np.array([1, -2, 3, -1])
print("Energia:", calc_energy(x_test)) # Esperado: 1+4+9+1 = 15
```

```python
def energy_per_band(wp, level=4):
    nodes = [n.path for n in wp.get_level(level, 'freq')]
    energies = []
    for n in nodes:
        energies.append(calc_energy(wp[n].data))
    return energies
```

Exercício

Crie um ruído branco e calcule a energia por banda para $L = 4$. A energia deve estar distribuída mais ou menos de forma uniforme? Teste.

Solução

```python
ruido = np.random.randn(16000)
wp_ruido = pywt.WaveletPacket(data=ruido, wavelet='db4',
    maxlevel=4)
energies_ruido = energy_per_band(wp_ruido)
print(energies_ruido)
```

Capítulo 6

RMS (Root Mean Square)

O **RMS** (Valor Quadrático Médio) é a raiz quadrada da média dos quadrados das amostras. Ele se relaciona com a energia, mas é normalizado pelo número de amostras, dando uma medida da amplitude efetiva do sinal. Fórmula: $RMS = \sqrt{\frac{1}{N}\sum_{n=1}^{N}|x[n]|^2}$

6.1 Implementação do RMS

```
def calc_rms(x):
    """Calcula o RMS do sinal x."""
    return np.sqrt(np.mean(np.square(x)))
```

```
print("RMS:", calc_rms(x_test)) # np.sqrt(15 / 4) = np.sqrt(3.75) =
    1.936
```

```
def rms_per_band(wp, level=4):
    nodes = [n.path for n in wp.get_level(level, 'freq')]
    rms_vals = []
    for n in nodes:
        rms_vals.append(calc_rms(wp[n].data))
    return rms_vals
```

Exercício

Como o tamanho do sinal (N) varia dependendo do nível, o RMS é melhor que a Energia Absoluta se compararmos duas sub-bandas de tamanhos diferentes?

Capítulo 7

ZCR (Taxa de Cruzamento por Zero)

A **Taxa de Cruzamento por Zero** (*Zero Crossing Rate* - ZCR) é o número de vezes que o sinal troca de sinal (positivo para negativo ou vice-versa). Sinais com mais altas frequências e ruídos tendem a ter um ZCR maior do que sinais suaves de baixa frequência.

7.1 Implementação do ZCR

```python
def calc_zcr(x):
    """Calcula a Taxa de Cruzamento por Zero."""
    # (x[:-1] * x[1:]) < 0 verifica as trocas de sinal
    zero_crosses = np.where(np.diff(np.signbit(x)))[0]
    return len(zero_crosses) / len(x)
```

```python
# Testando o ZCR
x_zcr_test = np.array([1, 1, -1, -1, 1, -1])
print("ZCR:", calc_zcr(x_zcr_test)) # 3 cruzamentos em 6 amostras
```

```python
def zcr_per_band(wp, level=4):
    nodes = [n.path for n in wp.get_level(level, 'freq')]
    zcr_vals = []
    for n in nodes:
        zcr_vals.append(calc_zcr(wp[n].data))
    return zcr_vals
```

Exercício

Crie um tom senoidal de 50 Hz e outro de 5000 Hz, ambos amostrados a 16000 Hz. Calcule o ZCR de cada um.

Solução

```python
t2 = np.linspace(0, 1, 16000)
seno_50 = np.sin(2 * np.pi * 50 * t2)
seno_5000 = np.sin(2 * np.pi * 5000 * t2)
print("ZCR 50Hz:", calc_zcr(seno_50))
print("ZCR 5000Hz:", calc_zcr(seno_5000))
```

Capítulo 8

Envelope via Transformada de Hilbert

A **Transformada de Hilbert** é fundamental para análises de contorno de onda. Intuitivamente, ela altera a fase do sinal em 90 graus. A combinação do sinal original (real) com a sua transformada de Hilbert (imaginária) nos dá o **sinal analítico**.

A magnitude desse sinal analítico nos fornece o **envelope** do sinal original, uma curva suave delineando os picos da onda.

Para caracterizar o envelope, extrairemos:

- **Média:** amplitude média do envelope.
- **Desvio Padrão (std):** variabilidade do envelope.
- **Curtose:** "achatamento"da distribuição das amostras do envelope (`scipy.stats.kurtosis`).

8.1 Implementação do Envelope e Descritores

```python
from scipy.signal import hilbert
from scipy.stats import kurtosis

def get_envelope(x):
    """Calcula o envelope usando a transformada de Hilbert."""
    analytic_signal = hilbert(x)
    amplitude_envelope = np.abs(analytic_signal)
    return amplitude_envelope
```

```python
def calc_envelope_features(x):
    """Retorna media, desvio padrao e curtose do envelope."""
    env = get_envelope(x)
    return np.mean(env), np.std(env), kurtosis(env)
```

```python
# Testando
env_m, env_s, env_k = calc_envelope_features(x_test)
print(f"Env Mean: {env_m:.3f}, Std: {env_s:.3f}, Kurtosis: {env_k
    :.3f}")
```

```python
def env_features_per_band(wp, level=4):
    nodes = [n.path for n in wp.get_level(level, 'freq')]
    feats = []
```

```
    for n in nodes:
        m, s, k = calc_envelope_features(wp[n].data)
        feats.append([m, s, k])
    return feats
```

Capítulo 9

Combinando Descritores por Sub-banda

Agora vamos juntar todos os descritores extraídos por sub-banda. Para cada sub-banda $i \in [1, 16]$, extrairemos: `[Energia, RMS, ZCR, Env_Mean, Env_Std, Env_Kurtosis]`.

Isso nos dará $16 \times 6 = 96$ características no total por segmento de áudio.

9.1 O Extrator Final

```python
def extract_all_features_from_signal(signal, wavelet='db4', level=4):
    wp = pywt.WaveletPacket(data=signal, wavelet=wavelet, mode='symmetric', maxlevel=level)
    nodes = [n.path for n in wp.get_level(level, 'freq')]

    feature_vector = []

    for n in nodes:
        data = wp[n].data
        # Extraindo caracter sticas base
        e = calc_energy(data)
        r = calc_rms(data)
        z = calc_zcr(data)

        # Extraindo caracter sticas de envelope
        env_m, env_s, env_k = calc_envelope_features(data)

        # Guardando o vetor da sub-banda
        sub_band_features = [e, r, z, env_m, env_s, env_k]
        feature_vector.extend(sub_band_features)

    return np.array(feature_vector)
```

```python
# Teste de extra  o
full_vector = extract_all_features_from_signal(signal)
print("Dimens o do Vetor de Caracter sticas:", full_vector.shape)
    # Esperado: (96,)
```

Capítulo 10

Documentação de Parâmetros

A reprodutibilidade é a base do trabalho científico. Todo experimento de extração de features **deve** ser acompanhado de um dicionário de configuração gravado junto aos dados extraídos (ex: em um arquivo JSON).

10.1 Dicionário de Configuração

```
import json

config = {
    "sampling_rate": 16000,
    "wavelet_family": "db4",
    "decomposition_level": 4,
    "padding_mode": "symmetric",
    "descriptors_per_band": ["Energy", "RMS", "ZCR", "Env_Mean", "
       Env_Std", "Env_Kurtosis"],
    "total_bands": 16,
    "total_features": 96
}

# Salvando a configura  o
with open("feature_extraction_config.json", "w") as f:
    json.dump(config, f, indent=4)
```

Capítulo 11

Brincadeiras Interativas

Vamos testar funções soltas para entender melhor os descritores. Rode os exemplos em um Jupyter Notebook ou num script Python.

```
# Brincadeira 1: Sinal constante vs Energia
c = np.ones(100)
print("Energia (100 amostras em 1):", calc_energy(c))
```

```
# Brincadeira 2: O ZCR e o sil ncio perfeito
z_sil = np.zeros(100)
print("ZCR do Sil ncio:", calc_zcr(z_sil)) # zero
```

```
# Brincadeira 3: Envelope de um impulso
imp = np.zeros(100)
imp[50] = 1.0
print("Curtose do envelope do Impulso:", calc_envelope_features(imp
    )[2])
```

```
# Brincadeira 4: Sub-bandas de um sinal puro
wp_pure = pywt.WaveletPacket(np.sin(2*np.pi*100*np.linspace
    (0,1,16000)), 'db4', maxlevel=4)
energies = energy_per_band(wp_pure)
# Qual banda tem mais energia para baixa frequ ncia?
print("Banda com maior energia:", np.argmax(energies))
```

```
# Brincadeira 5: M dia vs RMS
s_test = np.array([-1, 1, -1, 1])
print("M dia simples:", np.mean(s_test)) # 0
print("RMS:", calc_rms(s_test)) # 1.0
```

```
# Brincadeira 6: Envelope visual (usando Matplotlib)
import matplotlib.pyplot as plt
def plot_envelope():
    t = np.linspace(0, 0.1, 1000)
    sig = np.sin(2*np.pi*300*t) * np.exp(-10*t)
    env = get_envelope(sig)
    plt.plot(t, sig, label='Sinal')
    plt.plot(t, env, 'r', label='Envelope', linewidth=2)
    plt.legend()
```

```
10      # plt.show()
11  # plot_envelope()
```

Capítulo 12

Checklist do Mês 4

Ao finalizar este mês, certifique-se de que:

Definiu e justificou $L = 4$ como o nível inicial de decomposição.

Selecionou a wavelet de base (`db4`).

Implementou com sucesso a extração de **Energy**, **RMS** e **ZCR** para as 16 sub-bandas.

Utilizou `scipy.signal.hilbert` corretamente para encontrar o envelope, calculando **Média, Std e Curtose**.

Juntou todas essas 6 *features* por banda em um array plano de dimensão 96.

Possui o código versionado e configurado com a documentação de parâmetros gerada automaticamente.

Capítulo 13

Desafio Final do Mês 4

Desafio

Utilize duas amostras curtas reais do seu corpus (ex: 2 segundos de *Real* e 2 segundos de *Fake*). Carregue esses áudios usando a biblioteca `librosa`.

1. Extraia as características de ambos utilizando a função `extract_all_features_from_signal`.
2. Calcule a diferença absoluta entre os vetores de características (`np.abs(feat_real - feat_fake)`).
3. Identifique **qual** descritor ou qual **sub-banda** teve a maior diferença. Isso pode dar um *insight* inicial fenomenal sobre como o seu classificador vai encontrar os deepfakes no futuro!

```
# Dica de in cio para o Desafio:
import librosa
# y_real, _ = librosa.load("audio_real.wav", sr=16000)
# y_fake, _ = librosa.load("audio_fake.wav", sr=16000)
# feats_real = extract_all_features_from_signal(y_real)
# feats_fake = extract_all_features_from_signal(y_fake)
# diff = np.abs(feats_real - feats_fake)
# print("Maior diferen a no  ndice :", np.argmax(diff))
```